

Software Engineering Review

软件工程复习, 2025年一月, 2211030

- 选择, 80分
- 简答, 1题20分, 英文回答

1. Software and Software Engineering

软件的三个重要组成要素: Instructions, Data structures, Documentations

What is Software?

- Software is developed or engineered it is not manufactured in the classical sense. 不是工厂
- Software doesn't "wear out" but it does deteriorate. 生命周期、变坏
- Although the industry is moving toward component-based construction, most software continues to be custom-built. 定制化

Why must software change?

- Software must be *adapted* to meet the needs of new computing environments or technology, e.g. cloud computing
- Software must be *enhanced* to implement new business requirements, e.g. e-commerce
- Software must be *extended* to make it interoperable with other more modern systems or databases, e.g. AI framework
- Software must be *re-architected* to make it viable within a network environment, e.g. data center virtualization

Software Engineering Layers

- Tools
- Methods
- Process
- Aquality focus

==Process Framework Activities==

也是Process Model的五个步骤

- Communication
- Planning
- Modeling
- Construction
- Deployment

Essenceof Software Engineering Practice

1. *Understand the problem* (communication and analysis).
2. *Plan a solution* (modeling and software design).

3. Carry out the plan (code generation).
4. Examine result for accuracy (testing & quality assurance).

2. Process Models

Process Flow 有哪些

1. Linear process flow
2. Interactive process flow 迭代
3. Evolutionary process flow 进化
4. Parallel process flow

Process Model 有哪些

1. Waterfall Process Model 瀑布模型
 1. 以线性、顺序的方式完成每个阶段，每个阶段都有明确的目标和输出，并且需要完成当前阶段后才能进入下一阶段。
 2. 用户只能在开发结束后看到最终产品，缺乏灵活性，难以应对需求的变化
2. Prototyping Process Model 原型模型
 1. 用户可以参与开发过程，提高需求准确性。
 2. 通过快速构建系统原型，与用户交互和验证需求。
3. Spiral Process Model 螺旋模型
4. Unified Process Model 统一过程模型

3. Agility and Process

Agility 敏捷开发

- Effective (rapid and adaptive) response to change.
- Effective communication among all stakeholders.
- Drawing the customer onto the team.
- Organizing a team so that it is in control of the work performed. 有控制权
- Rapid, incremental delivery of software. 叠加式的交付

What is an Agile Process?

- Driven by customer descriptions of what is required (scenarios). 完全基于客户需求
- Customer feedback is frequent and acted on. 反馈
- Recognizes that plans are short-lived.
- Develops software iteratively with a heavy emphasis on construction activities.
- Delivers multiple 'software increments' as executable prototypes. 阶段性交付原型
- Adapts as project or technical changes occur. 适应性、灵活性强

Scrum Details 敏捷开发的详细信息

- Backlog Refinement Meeting 待办事项优化会议
- Sprint Planning Meeting 简短快速的计划会议
- Daily Scrum Meeting
- Sprint Review
- Sprint Retrospective

Framework

- Scrum Framework
- Extreme Programming (XP) Framework
- Kanban Framework

4. Recommended Process Model

Characteristics of Agile Process Models

1. Not suitable for large high-risk or mission critical projects.
2. Minimal rules and minimal documentation.
3. Continuous involvement of testers.
4. Easy to accommodate product changes.
5. Depends heavily on stakeholder interaction.
6. Easy to manage.
7. Early delivery of partial solutions.
8. Informal risk management.
9. Built-in continuous process improvement.

Incremental Model for Prototype Design 原型迭代

Spiral Model for Prototype Design 原型 循环的叠加

First Prototype Guidelines 指导原则

1. Transition from paper prototype to software design.
2. Prototype a user interface.
3. Create a virtual prototype.
4. Add input and output to your prototype.
5. Engineer your algorithms.
6. Test your prototype.
7. Prototype with deployment in mind.

Prototype Evaluation

1. Provide scaffolding when asking for prototype feedback.
2. Test your prototype on the right people.
3. Ask the right questions.
4. Be neutral when presenting alternatives to users.

5. Adapt while testing.
6. Allow the user to contribute ideas.

Software Release Maintenance 软件发布维护

- Maintenance
- Corrective maintenance 改正
- Adaptive maintenance 适配
- Perfective maintenance 改善
- Preventive maintenance 预防性维护

5. Human Aspects of Software Engineering

Traits of Successful Software Engineers

- Sense of individual responsibility.
- Acutely aware of the needs of team members and stakeholders (用户、公司决策者) .
- Brutally honest about design flaws and offers constructive criticism.
- Resilient under pressure. 在压力下具有弹性
- Heightened sense of fairness.
- Attention to detail.
- Pragmatic adapting software engineering practices based on the circumstances at hand. 根据手头的情况，务实地调整软件工程实践

Effective Software Team Attributes

- Sense of purpose. 使命感
- Sense of involvement. 参与感
- Sense of trust.
- Sense of improvement.
- Diversity of team member skill sets. 多样化

Agile Teams 敏捷团队

6. Principles that Guide Practice

Principles that Guide Process

1. Be agile.
2. Focus on quality at every step.
3. Be ready to adapt.
4. Build an effective team.
5. Establish mechanisms for communication and coordination. 建立沟通协调机制
6. Manage change. 管理变化
7. Assess risk. 评估风险

8. Create work products that provide value for others.

Principles that Guide Practice

1. Devide and conquer.
2. Understand the use of abstraction.
3. Strive for consistency.
4. Focus on the transfer of information.
5. Build software that exhibits effective modularity.
6. Look for patterns.
7. Use multiple viewpoints.
8. Some consumes your work products.

Communications Principles

1. Listen.
2. Prepare before you communicate.
3. Someone should faciltate the activity.
4. Face-to-face communication is best.
5. Take notes and document decision.
6. Strive for collaboration.
7. Stay focused, modularize your discussion.
8. If something is unclear, draw a picture.

Planning Principles

1. Understand the scope of the project.
2. Involve the customer in the planning activity.
3. Recognize tht planning is interactive.
4. Estimate based on what you know.
5. Consider risk as you define the plan.

Construction Principles - Coding

Preparation Principles: Before you write one line of code, be sure you:

1. Understand the problem to be solved.
2. Understand basic design principles and concepts.
3. Pick a programming language that meets the needs of the software to be built.
4. Select a programming environment that provides tools that will make your work easier.
5. Create a set of unit tests that will be applied once the component you code is completed.

7. Understanding Requirements

Requirements Engineering 需求工程

Non-functional Requirements 非功能需求

UML (Unified Modeling Language, 统一建模语言) 图是一种标准化的图形语言，用于建模软件系统的结构和行为。

8. Requirements Modeling - A Recommended Approach

Requirements Models

- **Scenario-based models** 基于场景
- **Class-oriented models** 面向类
- **Behavioral models** 基于行为
- **Data models** 基于数据
- **Flow-oriented models** 面向流

State Diagram 状态图、Activity Diagram 动作图、Swimlane Diagram 泳道图

9. Design Concepts

Mapping Requirements Model to Design Model 将需求模型映射到设计模型

- Component-Level Design 模块设计
- Interface Design 接口设计
- Architectural Design 系统架构设计
- Data/Class Design

Design Concepts

- Abstraction 抽象
- Architecture 架构
- Design Patterns 设计模式
- Separation of concerns 关注点分离
- Modularity 模块化
- Information Hiding 信息隐藏
- Functional independence 功能独立性
- Stepwise Refinement 逐步细化
- Refactoring 重构
- Design Classes 设计类

==Design Class Characteristics 设计类特征==

- Complete 完整
- Primitiveness 颗粒感
- High cohesion 高内聚
- Low coupling 低耦合

10. Architectural Design - A Recommended Approach

Architectural Styles

Each style describes a system category that encompasses:

1. **set of components** (for example: a database, computational modules) that perform a function required by a system.
2. **set of connectors** that enable “communication, coordination and cooperation” among components.
3. **constraints** that define how components can be integrated to form the system.
4. **semantic models (语义模型)** that enable a designer to understand the overall properties of a system by analyzing the known properties of its constituent parts.

Layered Architecture 层次化架构

Architectural Considerations

- **Economy** – software is uncluttered and relies on abstraction to reduce unnecessary detail.
- **Visibility** – Architectural decisions and their justifications should be obvious to software engineers who review.
- **Spacing** – Separation of concerns in a design without introducing hidden dependencies.
- **Symmetry** – Architectural symmetry implies that a system is consistent and balanced in its attributes.
- **Emergence** – Emergent, self-organized behavior and control are key to creating scalable, efficient, and economic software architectures.

11. Component-Level Design

12. User Experience Design

==User Experience Golden Rule==

- Place user in control
- Reduce user's memory load
- Make interface consistent

15. Quality Concepts

Software Quality

- Effective Process
- Useful Product

- Adding Value

16. Reviews - A Recommended Approach

17. Software Quality Assurance

Software Quality Assurance SQA

19. Software Testing - Component Level

Testing

20. Software Testing- Intergration Level
